

SOFTWARE ENGINEERING

Lab 1: Requirements Engineering & UML Use-Case Modelling

Problem Statement #41

OpenAPI Mock Server Generator

Consolidated Lab Submission

Prepared by	Arin Singh
SRN	PES1UG24CS079
Course	UE24CS341A : Software Engineering

<https://github.com/cosmocode-source/SE-Labs-PES1UG24CS079> - GitHub link

1. Requirements Table

Functional and non-functional requirements for the OpenAPI Mock Server Generator, reproduced from the project requirements document.

Functional Requirements

ID	Type	Requirement Description	Priority	Acceptance Criteria	Rationale	Use Case Ref
FR-001	Functional	The system should accept OpenAPI 3.0 specifications in YAML and JSON formats. According to specification mentioned	High	Pass: A valid OpenAPI 3.0 YAML or JSON file is successfully uploaded and we can read it. Fail: A supported valid file is rejected and error is mentioned.	The specification is required as input for generating the mock server.	UC-01
FR-002	Functional	The system shall validate the uploaded specification and display errors for invalid OpenAPI structures, paths, parameters, and schemas. The error should be mentioned specifically for the type of error.	High	Pass: An invalid specification is rejected with a clear error message and error location. Fail: The system generates endpoints from an invalid specification.	Validation prevents incorrect or unusable mock endpoints from being created.	UC-01
FR-003	Functional	The system shall dynamically generate mock HTTP endpoints matching the paths and HTTP methods defined in a valid specification.	High	Pass: Every defined path and HTTP method is available after starting the mock server. Fail: An undefined endpoint returns 200 OK.	Developers and testers need endpoints that correctly represent the specified API.	UC-01
FR-004	Functional	The system shall generate randomized JSON responses that comply with the response schemas and example values defined in the specification that are mentioned in the uploaded file or data.	High	Pass: Generated responses contain the required fields and correct JSON data types. Fail: A generated response violates the defined schema.	Schema-compliant responses allow development and testing before the real backend is available.	UC-01
FR-005	Functional	The system shall allow users to configure simulated response latency for mock endpoint responses.	High	Pass: When a latency value is configured, responses are delayed by approximately that duration. Fail: The configured delay is ignored.	Configurable latency allows QA Engineers to test application behaviour under slow-network or slow-server conditions.	UC-02

Non-Functional Requirements

ID	Type	Requirement Description	Priority	Acceptance Criteria	Rationale	Use Case Ref
NFR-001	Performance	The mock server shall sustain at least 1,000 mock API requests per second, with configured simulated latency accurate within ± 10 milliseconds.	High	Pass: A load test confirms that the server handles 1,000 requests per second and maintains the configured latency within ± 10 ms. Fail: The server cannot maintain the target request rate or latency accuracy.	The mock server must remain responsive and predictable during peak testing loads.	N/A
NFR-002	Security and Reliability	The system shall safely reject malformed or potentially harmful specification files without crashing or exposing internal server information.	High	Pass: Malformed and unsafe test files are rejected with controlled validation errors, and no stack traces or sensitive details are exposed. Fail: The server crashes, executes uploaded file content, or exposes internal details.	Safe file handling protects the mock server and its users from malicious or malformed input.	N/A

2. UML Use-Case Diagram

The diagram models the API Developer, QA Engineer, and API Client interactions with the OpenAPI Mock Server Generator, including mandatory «include» and conditional «extend» relationships.

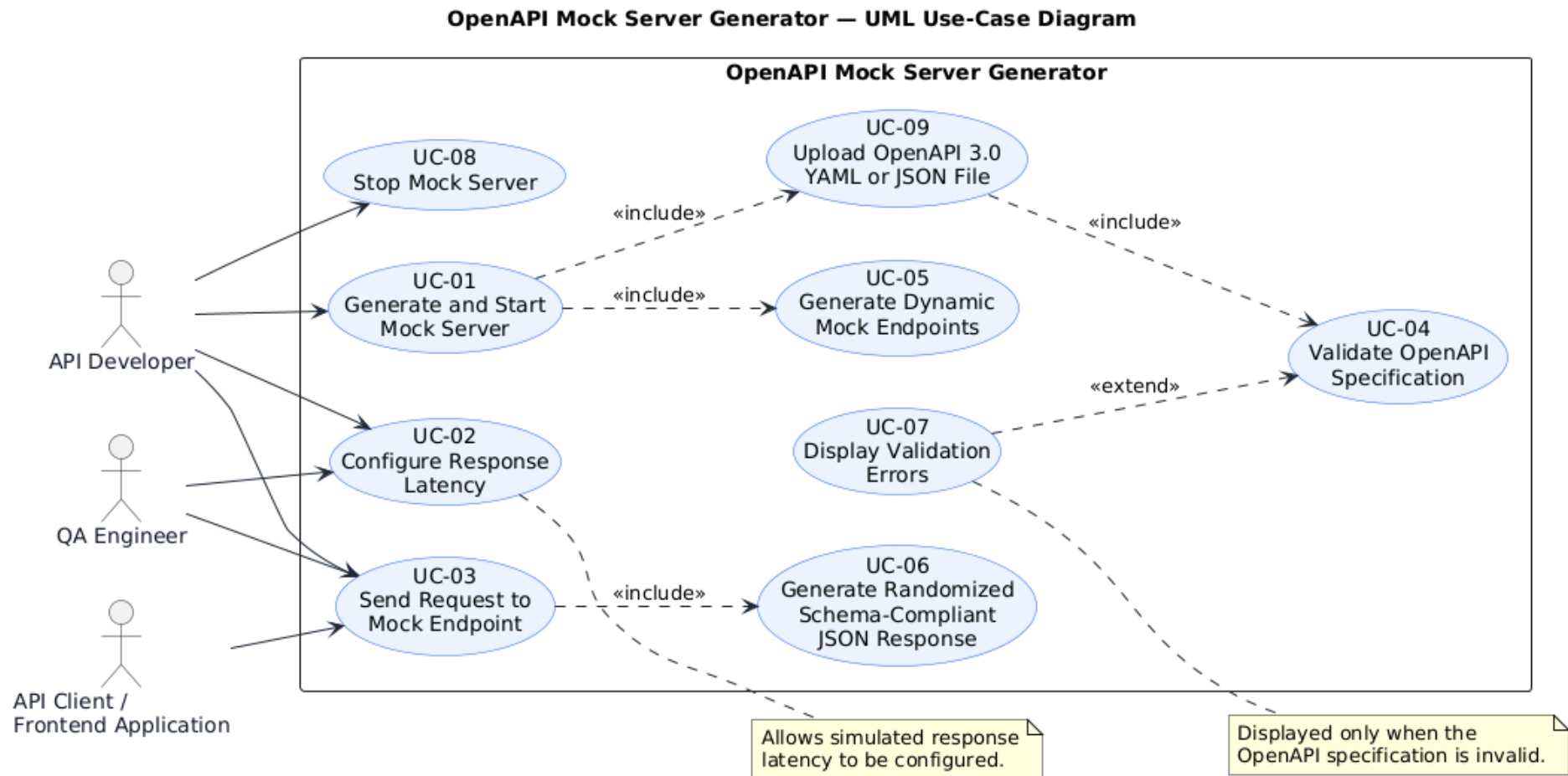


Figure 1. UML use-case diagram for the OpenAPI Mock Server Generator.

3. Use-Case Flow Document

Detailed flow specification for the core use case, generated from the project use-case Markdown document.

1. Use Case Name

Generate Mock Server (UC-01)

2. Summary / Description

This use case describes how a developer loads an OpenAPI specification file (in JSON or YAML format) and generates an active mock HTTP server that simulates the described endpoints, request validations, and response schemas.

3. Actors

- **Primary Actor:** Developer
- **Secondary Actor:** Frontend Application / API Client

4. Pre-conditions

1. The developer has a syntactically valid OpenAPI 3.0+ specification file (JSON or YAML format).
2. The OpenAPI Mock Server application is installed and running, or accessible via CLI/UI.

5. Post-conditions

- A mock HTTP server is instantiated and running on a target host/port.
- Endpoints corresponding to all paths and methods in the OpenAPI spec are reachable.
- Mock responses populated with schema data/examples are returned for valid requests.

6. Main Flow (Basic Flow)

1. The Developer provides the path or URL of an OpenAPI specification file to the system.
2. The System parses the specification file and validates its schema syntax.
3. The System builds internal routing tables for all defined paths, HTTP methods, parameters, and schemas.
4. The System starts the HTTP server on the configured port.
5. The System displays a summary of available mock endpoints and logs startup success.
6. The API Client sends HTTP requests to the generated mock endpoints.
7. The System validates incoming requests against parameter specifications and returns appropriate mock responses based on schema examples or type generation.

7. Alternative Flows

Alt-1: Spec Contains Examples

- At step 7, if response schemas include explicit example or examples objects, the System prioritizes serving those pre-defined examples over random generated dummy data.

Alt-2: Dynamic Parameter Validation Failure

- At step 7, if the request fails parameter validation (e.g. missing required header or path param), the System responds with a 400 Bad Request and validation error details.

8. Exception Flows**Exc-1: Invalid OpenAPI Specification**

1. At step 2, if the file fails OpenAPI syntax or schema validation:

- The System stops execution.
- The System displays specific parsing error logs indicating line numbers and structural errors.
- The use case ends unsuccessfully.

Exc-2: Port Already in Use

1. At step 4, if the target port is bound to another process:

- The System prompts an error message indicating port conflict.
- The System suggests an alternate port or allows user selection.